
aliases: - 04-listcomp-class

Day 4: 列表推导式 + lambda + 类/继承

读完约 10 分钟（分两次读也行）。

一、列表推导式（Day2 补课，今天应该先学这个）

你之前学过了循环：

```
squares = []
for x in range(1, 6):
    squares.append(x ** 2)
# squares = [1, 4, 9, 16, 25]
```

列表推导式是同一个逻辑的简写：

```
squares = [x ** 2 for x in range(1, 6)]
# 结果完全相同：[1, 4, 9, 16, 25]
```

语法拆解：

[对x做什么	for x in 可迭代对象	要不要过滤]
↑	↑	↑
x ** 2	for x in range(1, 6)	(可选)

带过滤的

```
# 原版循环
evens = []
```

```
for x in range(10):
    if x % 2 == 0:
        evens.append(x)

# 推导式
evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]
```

Java 对照：Stream API

```
// Java
List<Integer> squares = IntStream.range(1, 6)
    .map(x -> x * x)
    .boxed()
    .collect(Collectors.toList());
```

```
# Python 推导式
squares = [x ** 2 for x in range(1, 6)]
```

推导式更简洁，但不要写太复杂——**超过一层 for 就回去写普通循环。**

二、lambda：匿名函数

什么时候用

```
# 普通函数：有名字，可以反复调用
def square(x):
    return x ** 2

# lambda：没名字，用完就扔
lambda x: x ** 2
```

lambda 只在你需要一个「只用一次的小函数」时用。 能写 def 就写 def，别硬用 lambda。

语法

```
lambda 参数: 返回值表达式
```

只能写**一个表达式**（不能写多行、不能写 return、不能写 if 语句）。

配合 map/filter 使用

```
nums = [1, 2, 3, 4, 5]

# map: 对每个元素做操作
list(map(lambda x: x ** 2, nums))
# [1, 4, 9, 16, 25]

# filter: 保留满足条件的
list(filter(lambda x: x % 2 == 0, nums))
# [2, 4]
```

但说实话： 列表推导式比 map/filter 更 Pythonic:

```
# 不推荐
list(map(lambda x: x ** 2, nums))

# 推荐
[x ** 2 for x in nums]
```

结论：了解 lambda 是什么就行，日常能用推导式就用推导式。

三、类/继承：你有 Java 基础，这节很快

Python 的类 vs Java 的类

```
// Java
public class Dog {
    private String name;

    public Dog(String name) {
```

```
        this.name = name;
    }

    public void bark() {
        System.out.println(this.name + " 汪汪");
    }
}

Dog d = new Dog("旺财");
d.bark();
```

```
# Python
class Dog:
    def __init__(self, name):      # 构造函数，等于 Java 的 Dog(String name)
        self.name = name

    def bark(self):               # 方法，self 等于 Java 的 this
        print(f"{self.name} 汪汪")

d = Dog("旺财")
d.bark()
```

一一对应

Java	Python	说明
<code>class Dog {}</code>	<code>class Dog:</code>	定义类，Python 用冒号不用花括号
<code>public Dog(...)</code>	<code>def __init__(self, ...)</code>	构造函数
<code>this.name</code>	<code>self.name</code>	当前实例的成员变量
<code>new Dog()</code>	<code>Dog()</code>	创建实例，Python 不需要 new
<code>void bark()</code>	<code>def bark(self):</code>	方法，self 必须显式写在参数里

你唯一要适应的：`self` 必须显式写

```
class Counter:
    def __init__(self):
        self.count = 0      # 初始化成员变量

    def add(self, n):
        self.count += n     # 用 self.count 访问

    def show(self):
        print(f"当前计数: {self.count}")

c = Counter()
c.add(5)
c.add(3)
c.show()    # 当前计数: 8
```

Java 里 this 是可写可不写的（除非重名），Python 里 self 必须写。 这是最大的习惯差异。

继承

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        pass

class Dog(Animal):
    # ← 继承用括号，等于 Java 的 extends
    def speak(self):
        # 重写父类方法
        return f"{self.name} 汪汪"

class Cat(Animal):
    def speak(self):
        return f"{self.name} 喵喵"

d = Dog("旺财")
c = Cat("咪咪")
print(d.speak())  # 旺财 汪汪
print(c.speak())  # 咪咪 喵喵
```

跟 Java 一样： 继承 → 重写方法 → 多态。

当前级别许可

内容	状态
列表推导式	⚠️ 今天学完切片后可以碰，但只做简单推导式
lambda	⚠️ 了解即可，不要求手写熟练
map/filter	⚠️ 了解即可，日常用推导式代替
类/init/self	✅ 今天可以学（Java基础好，这节很快）
继承	✅ 今天可以学

相关笔记：[列表深入操作](#) · [推导式进阶](#) · [any/all/sorted](#)

桥接：把这个 Dog 换成你的 notes-api

你笔记里的 Dog 和你的 JsonNoteRepository，是**同一个模式**：

```
# —— 笔记里的例子 ——
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(f"{self.name} 汪汪")

# 创建对象
d = Dog("旺财")
# ^ 传字符串"旺财"给 __init__

# 使用方法
d.bark()
```

```
# —— 你项目里的代码 ——
class JsonNoteRepository:
    def __init__(self, filename):
        self.filename = filename

    def list_notes(self):
        # 读文件，返回笔记列表

# 创建对象
repo = JsonNoteRepository("notes.json")
# ^ 传字符串路径给 __init__

# 使用方法
notes = repo.list_notes()
```

完全一致的结构：

```
Dog("旺财")          → 类名(参数) = 造对象, 参数传给 __init__
    → __init__(self, "旺财")    → self.name = "旺财"

JsonNoteRepository("notes.json") → 类名(参数) = 造对象, 参数传给 __init__
    → __init__(self, "notes.json") → self.filename = "notes.json"
```

所以当你看到：

```
repo = JsonNoteRepository("notes.json")
service = NoteService(repo)
```

读作：

```
# 第1行：造一个"JSON仓库"对象，告诉它"你的文件是 notes.json"
repo = JsonNoteRepository("notes.json")

# 第2行：造一个"笔记服务"对象，把仓库塞给它
# NoteService 需要仓库才能干活——它不自己造仓库，而是外面传进来
service = NoteService(repo)
```

关键认知： 类名(参数) = 造一个对象，参数传给 `__init__`。不需要 `new` 关键字。

一句话串起来

```
模具（类）      = 配方（告诉你怎么造）
产品（对象）    = 实际造出来的东西
__init__        = "造的时候要准备什么"
self            = "这个产品自己"
对象.方法()     = "让这个产品做一件事"
```